

CS7CS3 Advanced Software Engineering Group Project

Functional and Technical Architecture

Project Name: Response to City Disaster

Group 11

Keshwith Pyla - 25352427

Sai Ruthwik Thummurugoti - 25337822

Bingi Sainath - 25347028

Varsha Shivalingaiah - 25359618

Krupaa Suresh - 25334718

Sharath Pradeep - 25344610

Rohan Rony Varughese - 25345146

Teniola Malomo - 25334817

Pratik Pramod Mattha - 25358525

Nakshatra Shrivastava - 25332635

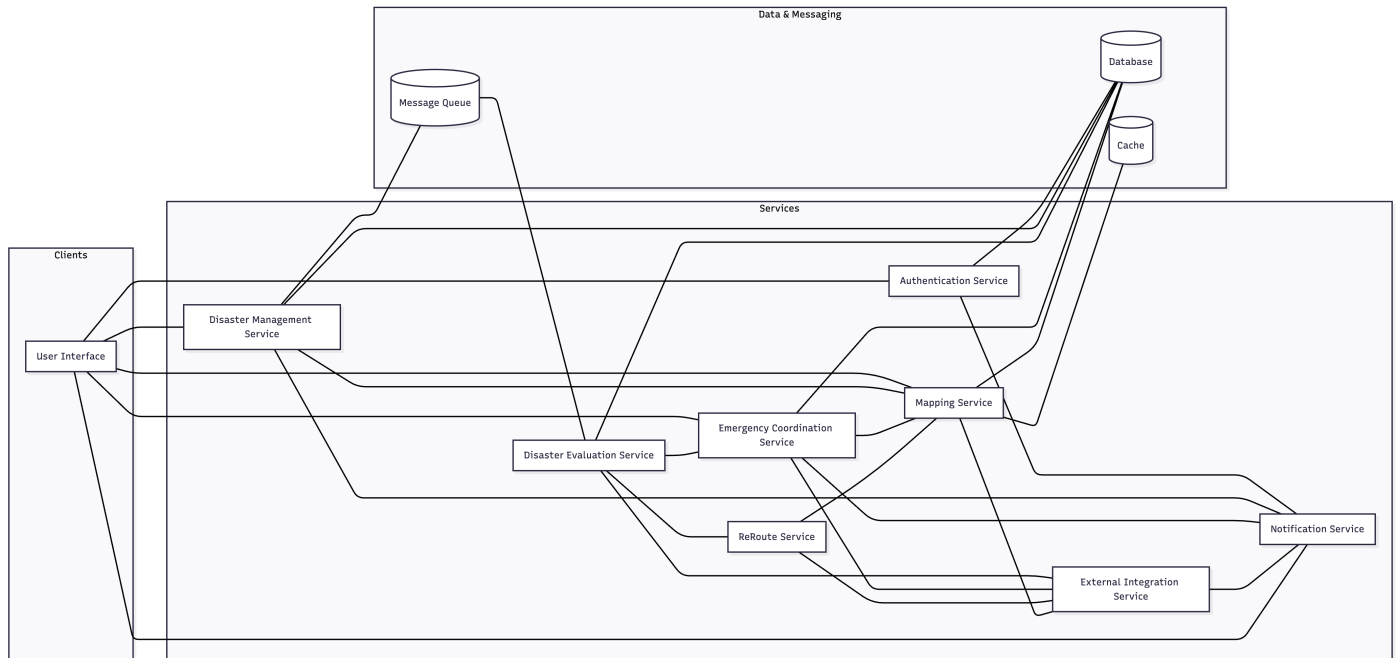
Table of Contents

Functional and Technical Architecture.....	1
1. Functional Architecture	3
1.1. Diagram	3
1.2. Component Descriptions	4
User Interface	4
Authentication Service.....	4
Disaster Report Service.....	4
Disaster Evaluation Service.....	4
Database	5
Notification Service	5
Mapping Service	5
Emergency Coordination Service.....	5
Re-route Service.....	6
Message Queue.....	6
Cloud Storage	6
External Integration Service.....	7
1.3. Component Interactions	8
Use Case 1: Login	8
Use Case 2: Signup.....	9
Use Case 3: Report Disaster.....	10
Use Case 4: Live Map	11
Use Case 5: Disaster Evaluation.....	12
Use Case 6: Deploy Services	13
Use Case 7: Re-Route Traffic.....	14
Use Case 8: Plan Evacuation	15
1.4. Component APIs	16
Authentication Service.....	16
Disaster Management Service	16
Disaster Evaluation Service	17
Database	18
Notification Service	19
Mapping Service	20
Emergency Coordination Service.....	21
Re-Route Service	23
Message Queue.....	24
Cloud Storage	24
External Integration Service.....	24
Cache	25
2. Technical Architecture.....	25
2.1. Diagram	25
Layer Overview:	26
2.2. Quality of Service Technical Requirements	27
Security Requirements.....	27
System Management.....	28
Client-side Management.....	31
Network Management.....	34
Server-side Management	36
Application Logic.....	38
Connectors	40

1. Functional Architecture

1.1. Diagram

The functional architecture of the Dublin City Disaster Response System follows a layered approach with clear separation of concerns:



The architecture consists of three main layers:

- Actor Layer: Citizens interact with the system through the User Interface
- Services Layer: Core business logic components including Disaster Management, Disaster Evaluation, Emergency Coordination, Authentication, Mapping, External Integration, and Notification services
- Data & Messaging Layer: Message Queue for asynchronous communication, Database for persistent storage, and Cloud for file storage

1.2. Component Descriptions

User Interface

The User Interface component provides the front-end presentation layer for all user interactions with the Dublin City Disaster Response System.

Responsibilities:

- Display interactive map views showing disasters, traffic, evacuation routes, and emergency services
- Provide registration and authentication forms for citizens and emergency response team members
- Enable disaster reporting with location selection, type categorization, severity rating, and media upload
- Show real-time notifications and alerts to users
- Display evacuation plans and safe routes for citizens
- Provide emergency response team dashboard for managing incidents and coordinating responses
- Support location permission requests and GPS integration
- Render responsive layouts for mobile and desktop devices

Authentication Service

The Authentication Service manages user identity verification and session management using OTP-based authentication.

Responsibilities:

- Generate secure 6-digit OTP codes for user authentication
- Validate mobile number formats (Irish mobile numbers)
- Send OTP codes via SMS integration
- Verify OTP codes and manage expiry timers (5-minute validity)
- Create and manage user sessions with secure tokens
- Differentiate between user roles (Citizens and Emergency Response Team)
- Handle pending approval status for Emergency Response Team members
- Log authentication events for audit trail
- Manage OTP resend functionality with cooldown periods (60 seconds)

Disaster Report Service

The Disaster Report Service is the core component responsible for processing disaster reports.

Responsibilities:

- Receive and validate disaster reports from citizens and emergency personnel
- Assign unique disaster IDs and updates database
- Triggers Disaster Evaluation Service
- Detect duplicate disaster reports and merge related incidents

Disaster Evaluation Service

The Disaster Evaluation Service performs automated severity assessment and impact analysis for reported disasters.

Responsibilities:

- Analyze disaster report details including type, location, severity, and supporting evidence
- Fetch supplementary data from external sources (weather, traffic, historical incidents)
- Calculate impact radius based on disaster characteristics
- Identify affected infrastructure including roads, buildings, and critical facilities
- Estimate population in affected areas using density data
- Determine required emergency response scale and resource levels
- Generate evaluation results with confidence scores
- Detect false alarms and duplicate reports
- Perform continuous re-evaluation as conditions change

- Trigger appropriate response workflows based on severity

Database

The Database component provides persistent storage for all system data including user accounts, disaster records, deployments, and historical information.

Responsibilities:

- Store user account information (credentials, roles, status, contact details)
- Maintain disaster records with complete details and timeline
- Track emergency service deployments and resource allocation
- Store evacuation plans and execution status
- Maintain audit logs for all critical operations
- Store pending registrations during OTP verification
- Support query operations for location-based searches
- Maintain historical incident data for analysis
- Ensure data integrity and GDPR compliance

Notification Service

The Notification Service handles all outbound communications to users including SMS, push notifications, and in-app alerts.

Responsibilities:

- Send OTP codes via SMS for authentication
- Dispatch evacuation alerts to affected populations
- Send push notifications for disaster updates and status changes
- Notify Emergency Response Team members of new incidents and assignments
- Alert citizens of disasters in their vicinity
- Send approval status notifications for Emergency Response Team registrations
- Provide multi-channel communication (SMS, push, in-app)
- Track message delivery status
- Support bulk notifications for evacuation scenarios
- Manage notification priorities and queuing

Mapping Service

The Mapping Service provides interactive map functionality, location services, and geographic data visualization for the entire system.

Responsibilities:

- Render base map layers (street, satellite, terrain views)
- Display disaster markers with severity color coding
- Show real-time traffic conditions and congestion levels
- Visualize evacuation routes and safe zones
- Display emergency service locations and movements
- Mark road closures and blocked areas
- Provide geocoding and reverse geocoding services
- Calculate optimal routes between locations
- Support map interactions (zoom, pan, marker selection)
- Integrate GPS location services
- Cache map tiles for offline functionality

Emergency Coordination Service

The Emergency Coordination Service manages the deployment and coordination of emergency response resources including ambulances, fire brigades, police, and rescue teams.

Responsibilities:

- Determine appropriate emergency services based on disaster type and severity
- Query available emergency units by type, location, and capabilities
- Calculate optimal resource allocation and dispatch strategies
- Create and manage deployment records with unique deployment IDs
- Track emergency unit status (Available, Dispatched, En Route, On Scene, Engaged)
- Monitor real-time GPS positions of deployed units
- Calculate and update estimated arrival times
- Handle additional resource requests from field personnel
- Manage resource recalls and redeployments
- Coordinate communication between command centre and field units

Re-route Service

The Re-route Service dynamically manages traffic flow around disaster zones through route optimization and traffic control measures.

Responsibilities:

- Identify affected road segments based on disaster impact zones
- Calculate alternative routes avoiding restricted areas
- Monitor real-time traffic conditions and congestion levels
- Compute detours optimized for travel time and capacity
- Reserve emergency corridors for response vehicles
- Update digital signage and traffic control systems
- Detect and respond to detour route congestion
- Handle multiple concurrent incidents with priority-based routing
- Provide route updates to navigation systems
- Restore normal traffic flow when roads are cleared

Message Queue

The Message Queue provides asynchronous communication infrastructure for decoupling system components and ensuring reliable message delivery.

Responsibilities:

- Queue disaster evaluation tasks for processing
- Ensure reliable message delivery with retry mechanisms
- Handle high-volume concurrent messages during emergencies
- Maintain message ordering where required
- Provide dead-letter queues for failed messages

Cloud Storage

The Cloud Storage component manages file storage for photos, videos, documents, and other media uploaded by users.

Responsibilities:

- Store disaster report photos and videos
- Save credential documents for Emergency Response Team registration
- Provide secure file upload and download mechanisms
- Generate signed URLs for secure file access
- Compress uploaded images automatically
- Scan uploaded files for malware
- Enforce file size limits and type restrictions
- Maintain file metadata and associations

External Integration Service

The External Integration Service manages connections to external data providers and third-party APIs for enriching disaster response capabilities.

Responsibilities:

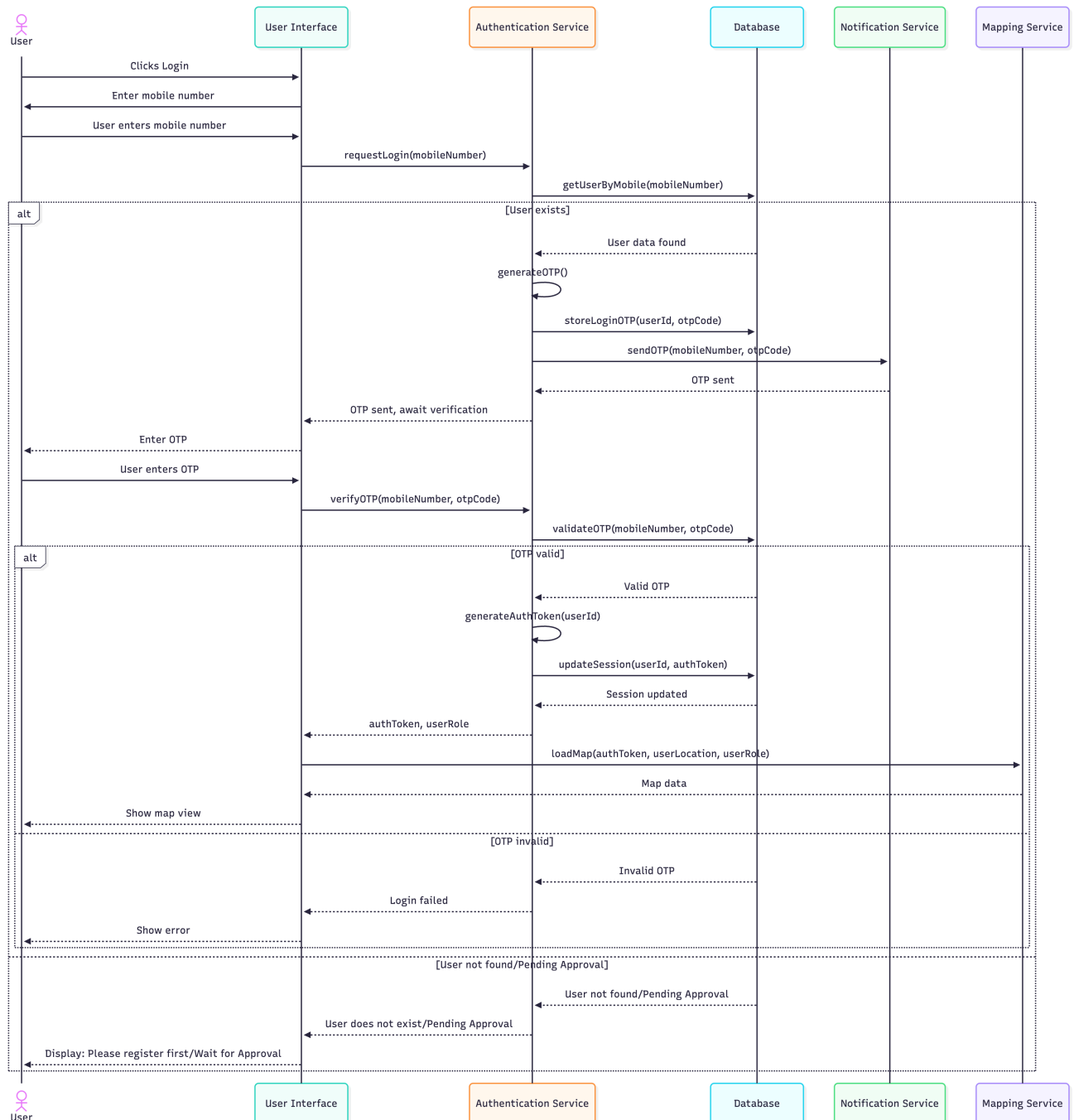
- Connect to traffic data APIs for live traffic conditions
- Fetch weather data from OpenWeatherMap or similar services
- Access CCTV and surveillance feeds where available
- Query population density and demographic databases
- Implement failover strategies for unavailable data sources
- Cache external data with appropriate expiry times
- Handle API rate limits and authentication
- Transform external data formats to internal schemas

1.3. Component Interactions

This section provides UML Sequence Diagrams for each use case, showing how the functional architecture components interact to achieve the use case goals.

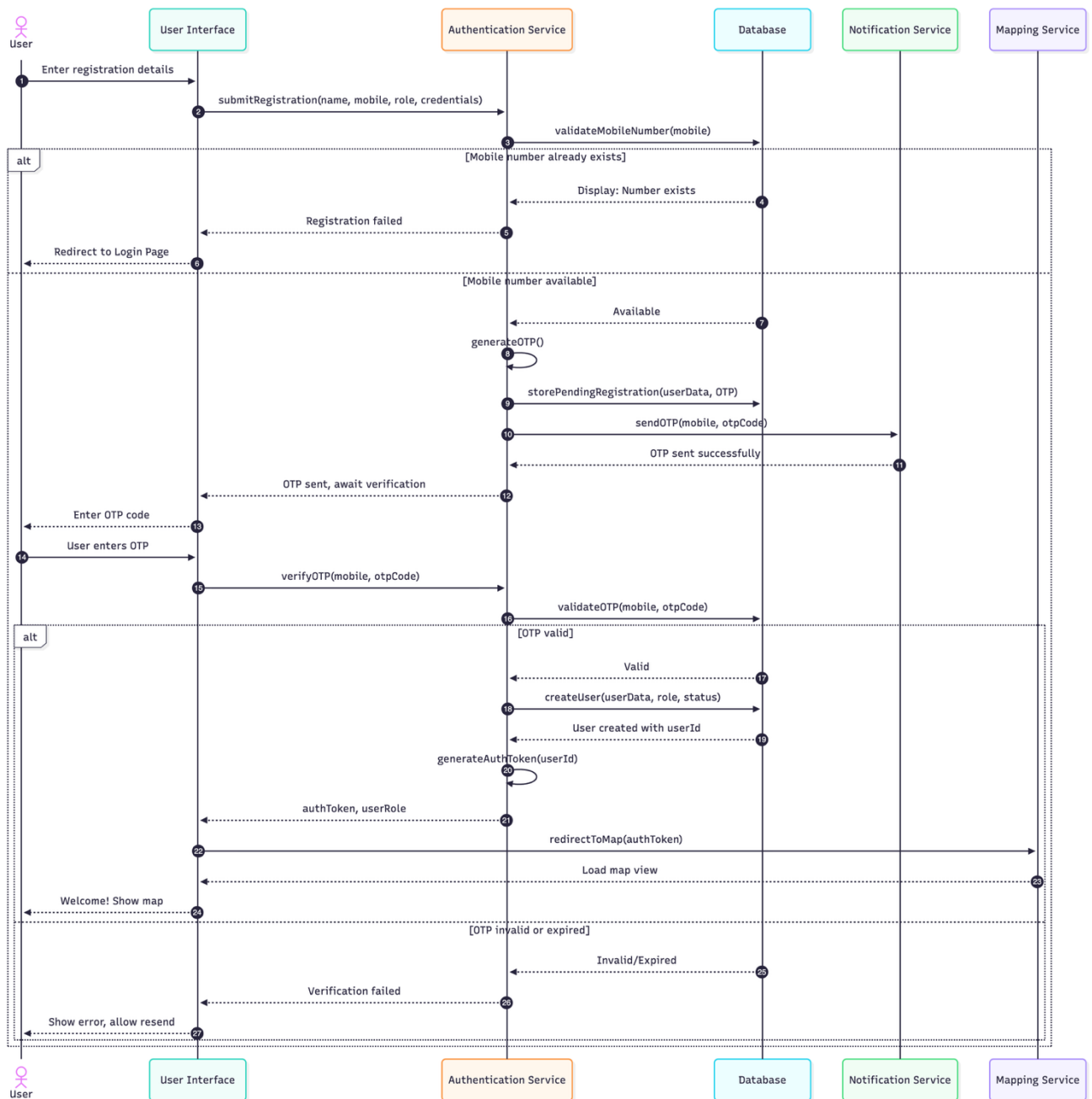
Use Case 1: Login

Sequence diagram showing the login process with OTP verification:



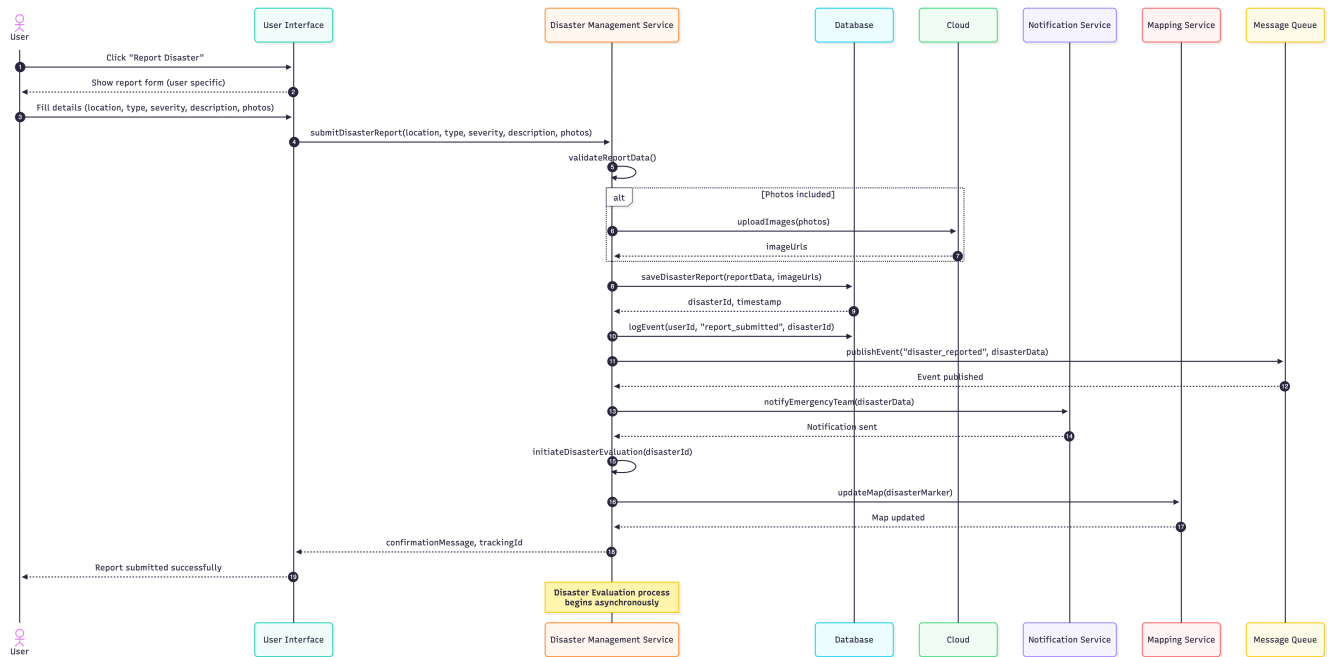
Use Case 2: Signup

Sequence diagram showing user registration with mobile number verification:



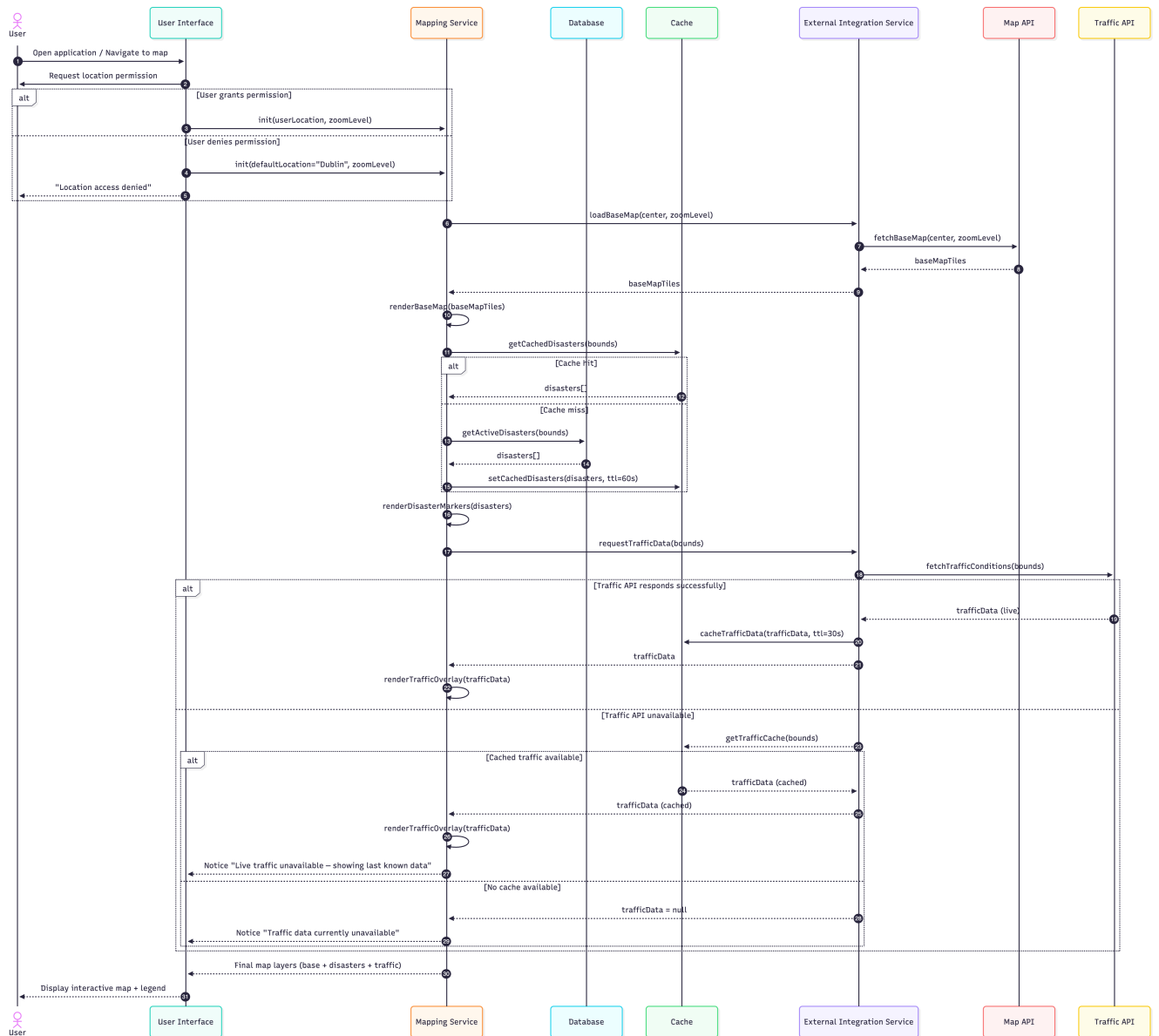
Use Case 3: Report Disaster

Sequence diagram showing disaster reporting and automatic evaluation trigger:



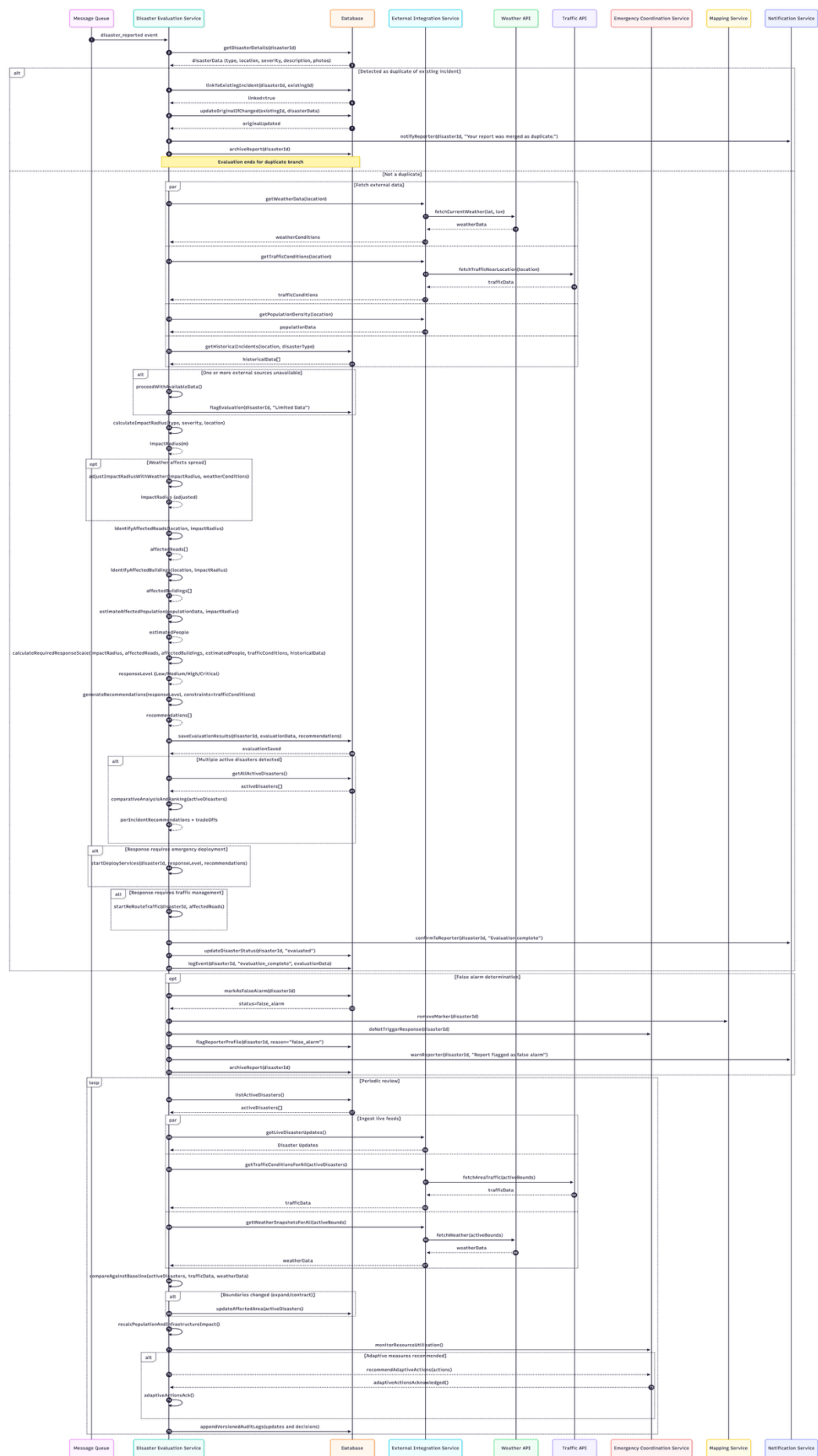
Use Case 4: Live Map

Sequence diagram showing live map data loading with real-time updates:



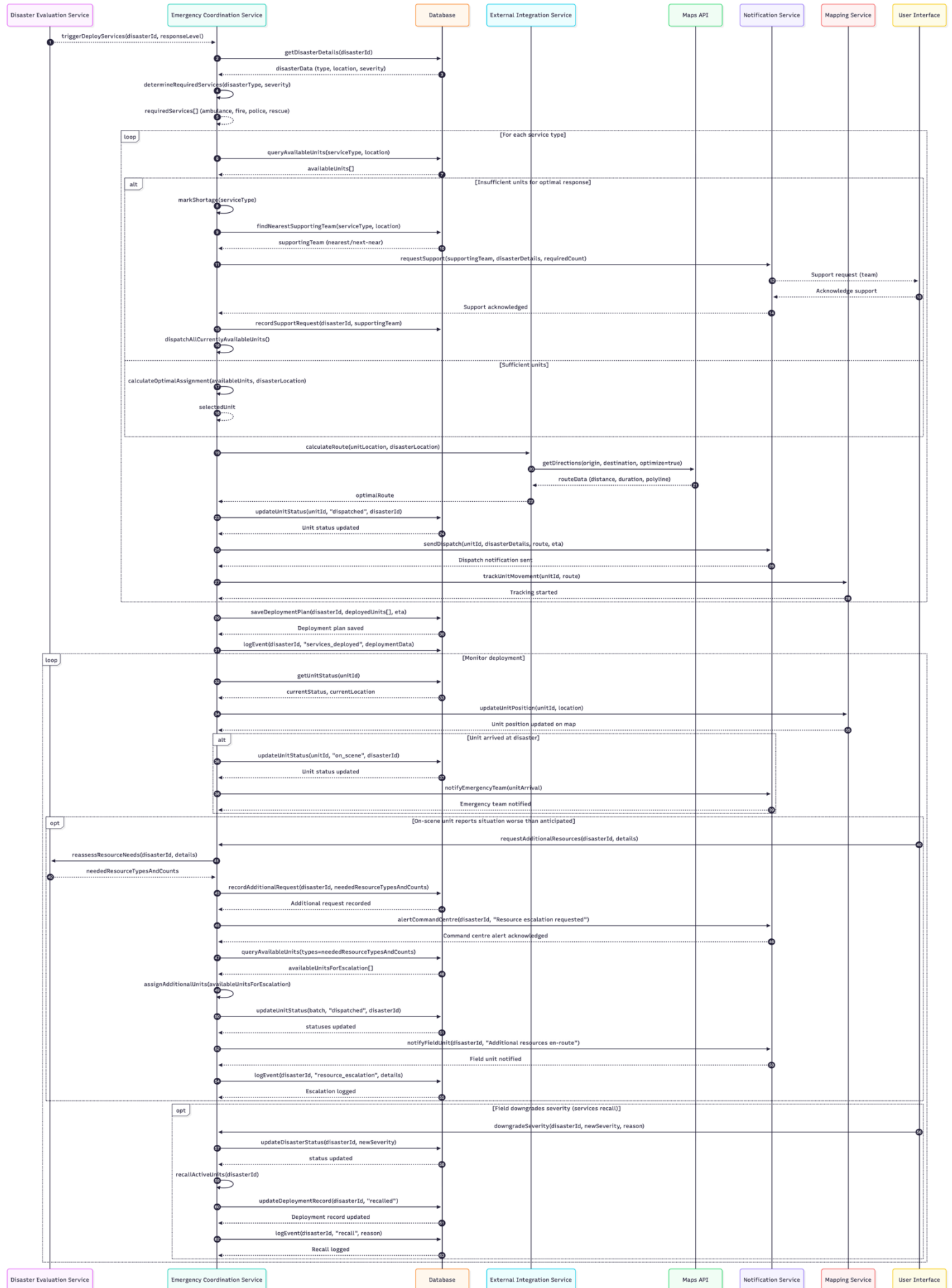
Use Case 5: Disaster Evaluation

Sequence diagram showing automated disaster severity assessment and response triggering:



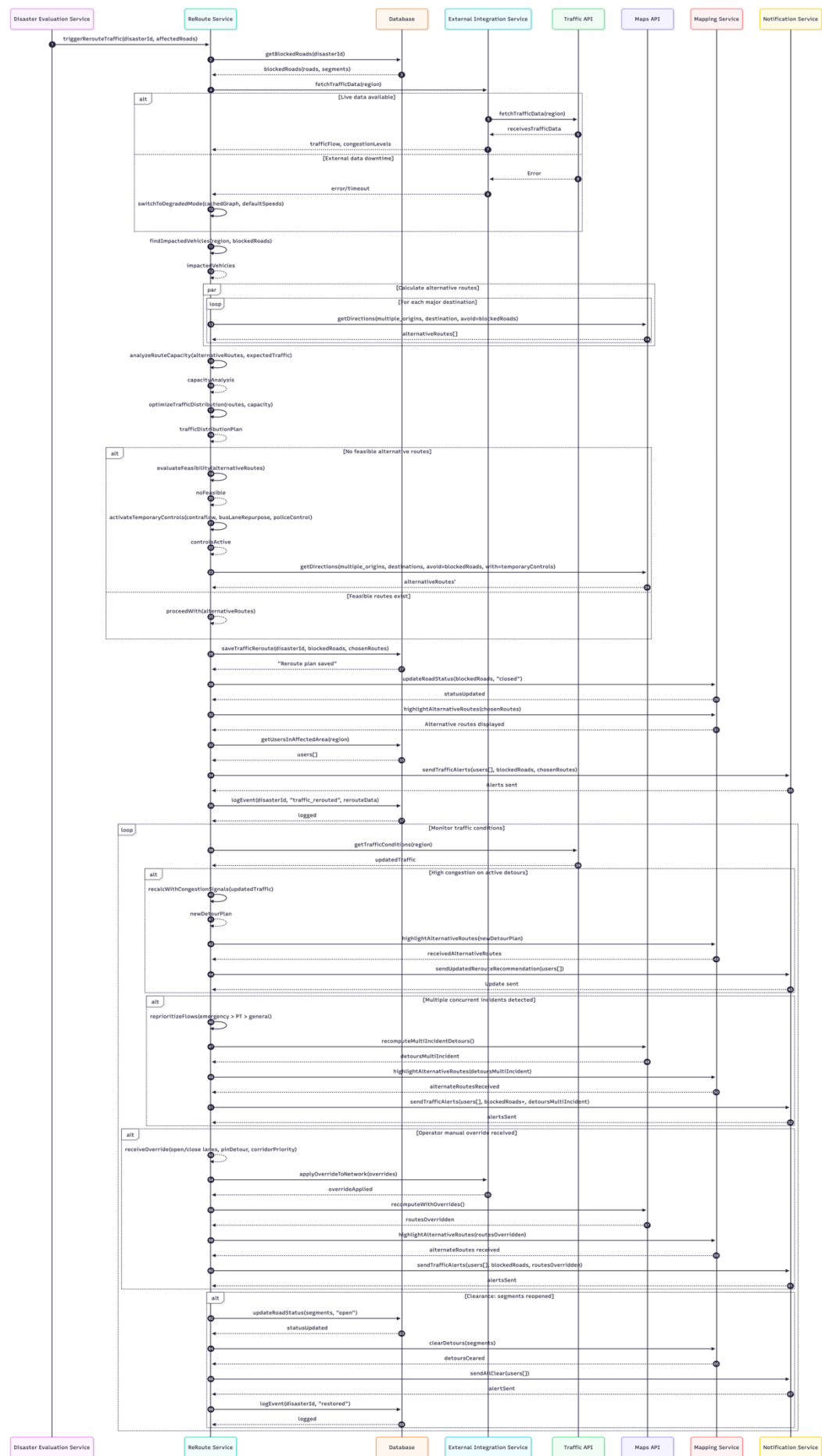
Use Case 6: Deploy Services

Sequence diagram showing emergency service deployment and coordination:



Use Case 7: Re-Route Traffic

Sequence diagram showing dynamic traffic rerouting around disaster zones:



Use Case 8: Plan Evacuation

Sequence diagram showing evacuation planning and coordination:



1.4. Component APIs

This section documents the Application Programming Interfaces (APIs) for each component as shown in the sequence diagrams. Each API includes its signature and description of responsibilities.

Authentication Service

- `void requestLogin(mobileNumber: string)`

Initiates login process by validating mobile number format and generating OTP.

- `User getUserByMobile(mobileNumber: string)`

Queries database to check if user exists with given mobile number.

- `string generateOTP()`

Creates cryptographically secure 6-digit random number for authentication.

- `boolean storeLoginOTP(userId: string, otpCode: string)`

Saves OTP code with 5-minute expiry timestamp in database.

- `boolean validateOTP(mobileNumber: string, otpCode: string)`

Checks if OTP matches and has not expired.

- `SessionToken generateAuthToken(userId: string)`

Creates secure session token for authenticated user.

- `boolean updateSession(userId: string, authToken: string)`

Stores session token and updates last login timestamp.

- `void submitRegistration(name: string, mobile: string, role: UserRole, credentials: Map)`

Processes new user registration with role-specific data validation.

- `boolean validateMobileNumber(mobile: string)`

Checks mobile number format and uniqueness in system.

- `boolean storePendingRegistration(userData: RegistrationData, otp: string)`

Temporarily stores registration data during OTP verification.

- `UserID createUser(userData: RegistrationData, role: UserRole, status: string)`

Creates new user account with specified role and approval status.

Disaster Management Service

- `DisasterID submitDisasterReport(location: Location, type: string, severity: string, description: string, photos: List<File>)`

Receives and validates disaster report, generates unique ID.

- `boolean validateReportData()`

Checks all required fields are present and valid.

- `List<string> uploadImages(photos: List<File>)`

Uploads disaster photos to cloud storage, returns URLs.

- `DisasterID saveDisasterReport(reportData: DisasterReport, imageUrls: List<string>)`

Persists disaster report in database with generated ID.

- `void logEvent(userId: string, eventType: string, disasterId: string)`

Records disaster reporting event in audit trail.

- `void notifyEmergencyTeam(disasterData: Disaster)`

Sends notification to Emergency Response Team about new disaster.

- `void initiateDisasterEvaluation(disasterId: string)`

Triggers disaster evaluation workflow asynchronously.

- `void updateMap(disasterMarker: DisasterMarker)`

Adds disaster marker to live map interface.

Disaster Evaluation Service

- `void triggerDisasterEvaluation(disasterId: string, triggerSource: string)`

Initiates automated disaster severity assessment process.

- `DisasterData getDisasterDetails(disasterId: string)`

Retrieves complete disaster report from database.

- `ExternalData fetchSupplementaryData(location: Location, radius: number)`

Gathers external data (weather, traffic, historical incidents) for location.

- `number determineImpactRadius(disasterType: string, severity: string)`

Calculates geographic radius of disaster impact.

- `List<Road> identifyAffectedRoads(location: Location, radius: number)`

Determines which roads fall within impact zone.

- `number estimateAffectedPopulation(location: Location, radius: number)`

Calculates estimated people in affected area.

- `ResponseScale calculateResponseScale()`

Determines required emergency response level.

- `EvaluationResult generateEvaluationResults()`

Compiles complete evaluation with recommendations.

- `void saveEvaluationResults()`

Persists evaluation results in database.

- `boolean checkDuplicates(location: Location, disasterType: string, timeWindow: number)`

Detects similar reports in same location and timeframe.

- `void markAsDuplicate(disasterId: string, originalDisasterId: string)`

Links duplicate report to original disaster.

- `void updateDisasterConfidence(originalDisasterId: string)`

Increases confidence score when duplicate confirms original.

- `void notifyReporter()`

Sends notification to user who reported disaster.

- `boolean detectFalseAlarm(disasterData: Disaster, externalData: ExternalData)`

Analyzes report against external data to identify false alarms.

- `void markAsFalseAlarm(disasterId: string)`

Updates disaster status and removes from active incidents.

- `void flagReporter(reporterId: string)`

Tracks false report rate for user accountability.

- `void notifyFalseAlarmReporter()`

Sends warning to user about false alarm.

- `void triggerDeployServices(disasterId: string, requiredServices: List<ServiceType>)`

Initiates emergency service deployment workflow.

- `void triggerReRouteTraffic(affectedRoads: List<Road>, detourRoutes: List<Route>)`

Initiates traffic rerouting workflow.

- `void triggerPlanEvacuation(evacuationZones: List<Zone>, estimatedPopulation: number)`

Initiates evacuation planning workflow.

Database

- `User getUserByMobile(mobileNumber: string)`

Queries users table for record matching mobile number.

- `boolean validateMobileNumber(mobileNumber: string)`

Checks if mobile number exists in database.

- `boolean updateSession(userId: string, authToken: string)`

Creates or updates session record.

- `UserID createUser(userData: RegistrationData, role: UserRole, status: string)`

Inserts new user record into users table.

- DisasterID saveDisasterReport(reportData: DisasterReport, imageUrls: List<string>)

Stores disaster report in disasters table.

- Disaster getDisasterDetails(disasterId: string)

Retrieves complete disaster record by ID.

- void saveEvaluationResults(disasterId: string, results: EvaluationResult)

Stores evaluation results linked to disaster.

- void updateDisasterConfidence(disasterId: string, confidenceScore: number)

Updates confidence level for disaster record.

- void flagReporter(reporterId: string)

Increments false report counter for user.

- List<Disaster> getActiveDisasters(bounds: GeographicBounds)

Queries active disasters within geographic bounds.

- List<EmergencyUnit> queryAvailableUnits(serviceType: string, location: Location)

Finds available units by type near location.

- DeploymentID createDeploymentRecord(disasterId: string, deploymentDetails: Deployment)

Creates deployment record in database.

- void updateUnitStatus(unitId: string, status: string)

Updates emergency unit status.

- List<Road> getRoadNetwork(location: Location, radius: number)

Retrieves road segments within radius.

- void updateRoadStatus(roadSegments: List<Road>, status: string)

Marks roads as closed or restricted.

- PopulationData getPopulationDensity(zones: List<Zone>)

Retrieves population data for evacuation zones.

- List<Shelter> getShelterCapacity(shelters: List<Shelter>)

Gets available capacity at shelters.

- List<TransportUnit> getAvailableTransport(location: Location)

Queries available transport resources.

- EvacuationID saveEvacuationPlan(evacuationPlan: EvacuationPlan)

Stores complete evacuation plan.

Notification Service

- boolean sendOTP(mobileNumber: string, otpCode: string)

Sends OTP via SMS to mobile number.

- void notifyEmergencyTeam(disasterData: Disaster)

Alerts Emergency Response Team of new disaster.

- void notifyReporter()

Sends notification to disaster reporter.

- void notifyFalseAlarmReporter()

Sends warning about false alarm to reporter.

- void sendDispatchNotification(unitId: string, deploymentData: Deployment)

Notifies emergency unit of dispatch assignment.

- void broadcastEvacuationAlerts(zones: List<Zone>, message: string)

Sends evacuation alerts to all users in zones.

- void sendEvacuationNotification(userId: string, evacuationDetails: EvacuationPlan)

Sends personalized evacuation instructions.

Mapping Service

- Map loadMap(authToken: string, userLocation: Location, userRole: UserRole)

Initializes map based on user authentication and location.

- Map redirectToMap(authToken: string)

Redirects user to map view after successful signup.

- Location initializeLocation(zoomLevel: number)

Sets up map view with user location or default.

- Location initDefaultLocation(city: string, zoomLevel: number)

Centers map on default city coordinates.

- MapData loadBaseMap(center: Location, zoomLevel: number)

Loads base map tiles for display.

- List<MapFile> fetchBaseMapTiles(center: Location, zoomLevel: number)

Retrieves map tiles from external API.

- void renderBaseMap(baseMapFiles: List<MapFile>)

Displays base map layer on interface.

- List<Disaster> getCachedDisasters(bounds: GeographicBounds)

Gets cached disaster markers for quick loading.

- List<Disaster> getActiveDisasters(bounds: GeographicBounds)

Queries active disasters in viewport.

- void setCachedDisasters(disasters: List<Disaster>, ttl: number)

Updates disaster marker cache.

- void renderDisasterMarkers(disasters: List<Disaster>)

Places disaster markers on map.

- TrafficData requestTrafficData(bounds: GeographicBounds)

Fetches current traffic conditions.

- TrafficData getTrafficCache(bounds: GeographicBounds)

Checks cache for recent traffic data.

- void cacheTrafficData(trafficData: TrafficData, ttl: number)

Stores traffic data in cache.

- void renderTrafficData(trafficData: TrafficData)

Displays traffic layer with color coding.

- void updateMap(disasterMarker: DisasterMarker)

Adds new disaster marker to live map.

- Route calculateOptimalRoutes(origin: Location, destination: Location)

Computes optimal route for emergency units.

- Route calculateRoute(origin: Location, destination: Location, constraints: RouteConstraints)

Calculates route considering closures and traffic.

- void updateMapClosures(closedRoads: List<Road>)

Marks closed roads on map.

- void updateMapDetours(detourRoutes: List<Route>)

Displays detour routes on map.

- List<Route> calculateEvacuationRoutes(zones: List<Zone>, shelters: List<Shelter>)

Computes optimal evacuation paths.

- void updateMapWithEvacuationRoutes(routes: List<Route>, shelters: List<Shelter>)

Displays evacuation routes on map.

- List<Shelter> identifySafeShelters(area: GeographicArea)

Finds safe shelters outside impact zone.

- void triggerDeployServices(disasterId: string, requiredServices: List<ServiceType>)

Initiates emergency service deployment workflow.

- Disaster getDisasterDetails(disasterId: string)

Retrieves disaster details for deployment planning.

- List<ServiceType> determineRequiredServices(disasterType: string, severity: string)

Identifies which services are needed.

- List<EmergencyUnit> queryAvailableUnits(serviceType: string, location: Location)

Finds available units by type near disaster.

- List<Route> calculateOptimalRoutes(unitLocation: Location, disasterLocation: Location)

Computes best routes for each unit.

- void assignUnitsToDisaster(disasterId: string, units: List<EmergencyUnit>, routes: List<Route>)

Assigns units to disaster with routes.

- DeploymentID createDeploymentRecord(disasterId: string, deploymentDetails: Deployment)

Creates deployment record in database.

- void updateUnitStatus(unitId: string, status: string)

Updates unit status to Dispatched/En Route/On Scene.

- void sendDispatchNotification(unitId: string, deploymentData: Deployment)

Notifies unit of dispatch assignment.

- void trackDeployedUnits(deploymentId: string)

Monitors real-time unit locations.

- void updateUnitLocation(unitId: string, location: Location)

Updates unit GPS position.

- void handleResourceRequest(unitId: string, additionalNeeds: string)

Processes field request for more resources.

- void deployAdditionalUnits(disasterId: string, additionalUnits: List<EmergencyUnit>)

Dispatches supplementary resources.

- void triggerPlanEvacuation(disasterId: string, affectedZones: List<Zone>, estimatedPopulation: number)

Initiates evacuation planning workflow.

- List<Zone> determineEvacuationZones(location: Location, impactRadius: number)

Identifies zones requiring evacuation.

- number identifyPopulationInZones(zones: List<Zone>)

Calculates population in evacuation zones.

- `List<Route> calculateEvacuationRoutes(zones: List<Zone>, shelters: List<Shelter>)`

Computes safe evacuation paths.

- `List<Shelter> identifySafeShelters(zones: List<Zone>, capacity: number)`

Finds shelters with sufficient capacity.

- `TransportNeeds estimateTransportNeeds(population: number, zones: List<Zone>)`

Calculates required transport resources.

- `List<TransportUnit> allocateTransportResources(transportNeeds: TransportNeeds)`

Assigns buses and vehicles for evacuation.

- `EvacuationPlan createEvacuationPlan(zones: List<Zone>, routes: List<Route>, shelters: List<Shelter>, transport: List<TransportUnit>)`

Generates complete evacuation plan.

- `EvacuationID saveEvacuationPlan(evacuationPlan: EvacuationPlan)`

Stores evacuation plan in database.

- `void broadcastEvacuationAlerts(zones: List<Zone>, evacuationPlan: EvacuationPlan)`

Sends alerts to all users in evacuation zones.

- `void updateMapWithEvacuationRoutes(routes: List<Route>, shelters: List<Shelter>)`

Displays evacuation info on map.

- `void trackEvacuationProgress(evacuationId: string)`

Monitors evacuation completion status.

- `void updateZoneStatus(zoneId: string, status: string)`

Updates evacuation progress for zone.

Re-Route Service

- `void triggerReRouteTraffic(disasterId: string, affectedRoads: List<Road>)`

Initiates traffic rerouting workflow.

- `Disaster getDisasterDetails(disasterId: string)`

Retrieves disaster location and impact data.

- `List<Road> identifyAffectedRoads(location: Location, radius: number)`

Determines roads within impact zone.

- `void markRoadsClosed(roadSegments: List<Road>)`

Updates road status to closed in database.

- `List<Vehicle> identifyAffectedVehicles(affectedArea: GeographicArea)`

Finds vehicles with routes through impact zone.

- `TrafficData fetchLiveTrafficData(area: GeographicArea)`

Gets real-time traffic conditions from external API.

- `List<Route> computeAlternativeRoutes(vehicle: Vehicle, closedRoads: List<Road>, liveTraffic: TrafficData)`

Calculates detour routes avoiding closures.

- `boolean validateRouteCapacity(alternativeRoutes: List<Route>)`

Checks if detours can handle traffic volume.

- `void dispatchRouteUpdates(vehicles: List<Vehicle>, newRoutes: List<Route>)`

Sends new navigation to affected vehicles.

- `void updateMapClosures(closedRoads: List<Road>)`

Marks road closures on live map.

- `void updateMapDetours(detourRoutes: List<Route>)`

Displays detour routes on map.

- `void monitorTrafficFlow(affectedArea: GeographicArea)`

Continuously tracks traffic in rerouted areas.

- `boolean detectDetourCongestion(detourRoutes: List<Route>)`

Identifies congestion on detour routes.

- `List<Route> recalculateRoutes(congestedDetour: Route)`

Computes new detours when congestion detected.

Message Queue

- `void publishEvent(eventType: string, disasterData: Disaster)`

Publishes disaster event for asynchronous processing.

Cloud Storage

- `List<string> uploadImages(photos: List<File>)`

Uploads photos to cloud storage and returns URLs.

External Integration Service

- `List<MapFile> fetchBaseMapTiles(center: Location, zoomLevel: number)`

Retrieves map tiles from external map API.

- `TrafficData fetchTrafficConditions(bounds: GeographicBounds)`

Gets current traffic data from traffic API.

- `WeatherData fetchWeatherData(location: Location)`

Retrieves weather conditions from weather API.

- `TrafficData fetchTrafficData(location: Location)`

Gets traffic conditions near disaster location.

- `PopulationData fetchPopulationDensity(area: GeographicArea)`

Retrieves population density data for area.

- `TrafficData fetchLiveTrafficData(area: GeographicArea)`

Gets real-time traffic conditions.

- `PopulationData fetchPopulationDensity(zones: List<Zone>)`

Gets population data for evacuation zones.

Cache

The Cache component provides temporary storage for frequently accessed data to improve performance.

- `List<Disaster> getCachedDisasters(bounds: GeographicBounds)`

Retrieves cached disaster markers for map area.

- `TrafficData getTrafficCache(bounds: GeographicBounds)`

Gets cached traffic data if available and not expired.

- `void cacheTrafficData(trafficData: TrafficData, ttl: number)`

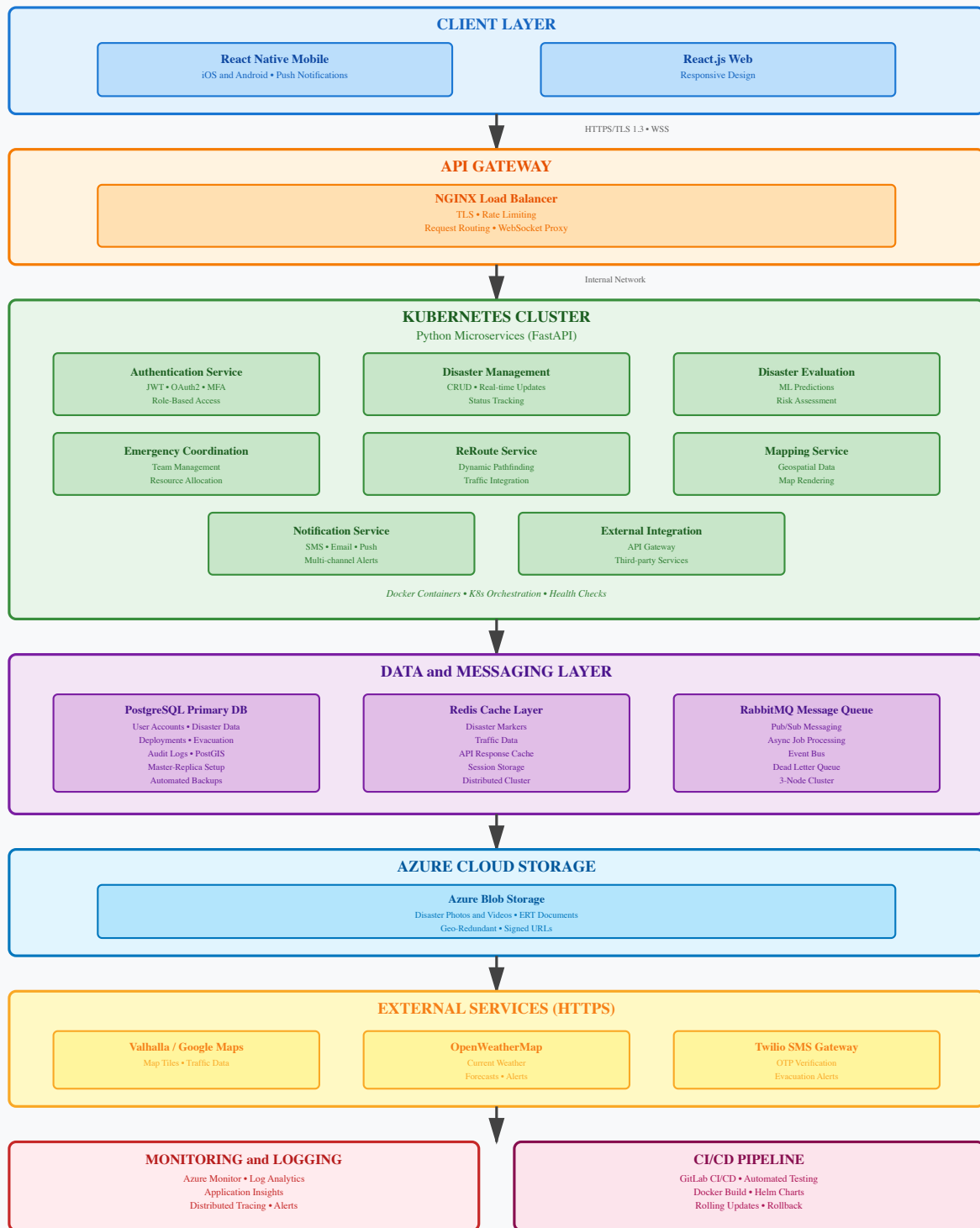
Stores traffic data in cache with 1 min expiry.

2. Technical Architecture

2.1. Diagram

The technical architecture shows deployment, infrastructure, and technology stack:

Disaster Management System Architecture



Layer Overview:

- Client Layer:** React Native mobile applications (iOS/Android) and React.js web applications with responsive design

- **API Gateway:** NGINX load balancer with TLS encryption, rate limiting, request routing, and WebSocket proxy capabilities
- **Kubernetes Cluster:** Python microservices built with FastAPI, containerized with Docker and orchestrated by Kubernetes
- **Data & Messaging Layer:** PostgreSQL (with PostGIS), Redis cache cluster, and RabbitMQ message queue
- **Azure Cloud Storage:** Blob storage for media files and documents with geo-redundancy
- **External Services:** Integration with Valhalla/Google Maps, OpenWeatherMap, and Twilio SMS Gateway

Supporting Infrastructure:

- **Monitoring & Logging:** Azure Monitor, Log Analytics, Application Insights with distributed tracing
- **CI/CD Pipeline:** GitLab CI/CD with automated testing, Docker builds, and rolling updates

2.2. Quality of Service Technical Requirements

Security Requirements

Security Requirement	Impact on Project and how the Technical Architecture will address the impact
Encryption - Do transactions need to be encrypted? - Level of encryption? (e.g., 40-bit encryption in US)	Yes, all transactions require encryption. Implementation: • TLS 1.3 for all HTTPS connections between clients and API gateway • 256-bit AES encryption for data at rest in PostgreSQL and Azure Blob Storage • TLS 1.2+ for service-to-service communication within Kubernetes cluster • JWT tokens encrypted with RS256 algorithm (2048-bit RSA keys) Architecture Impact: NGINX configured with strong cipher suites, certificate management with Let's Encrypt, automatic certificate rotation
User Identification - email/otp, cookies, application-level? - Existing customer database that should be used to identify online visitors?	Multi-layered authentication system. Implementation: • Primary: Mobile number/OTP verification • JWT tokens for session management • Role-based access control (RBAC): Citizen, Emergency Responder, Coordinator, Admin Architecture Impact: Dedicated Authentication Service microservice, secure token storage in mobile apps (Keychain/Keystore)
Access to data - Do you need to restrict access to parts of the site?	Yes, strict access controls required. Implementation: • API-level authorization middleware checking JWT roles on every request

- What privacy rules should be applied to information provided by users	• Data access restrictions: Citizens see only their own reports; ERTs see assigned disasters; Coordinators see all data in their region; Admins have full access • Privacy compliance: GDPR-compliant data handling, user consent management, right to deletion • PII encryption: Personal data (phone numbers, name, email) encrypted in database • Data minimization: Collect only necessary information for disaster response Architecture Impact: Authorization decorators in FastAPI, audit logging for all data access
What are the legal requirements and policies for auditing content, changes and transactions?	Comprehensive audit trail required for accountability. Implementation: • Audit logs stored in dedicated PostgreSQL table with append-only permissions • Logged events: User authentication, data modifications, disaster status changes, deployment actions • Log retention: 7 years to comply with government regulations • Tamper-proof logs: Cryptographic hashing (SHA-256) of log entries • Real-time log streaming to Azure Log Analytics for analysis Architecture Impact: Centralized logging middleware, immutable audit tables, Azure Monitor integration for compliance reporting
Do you plan to use a secure demilitarised zone into which your project server code could be placed?	Yes, DMZ architecture implemented. Implementation: • DMZ subnet hosting NGINX load balancer with public internet access • Application tier (Kubernetes cluster) in private subnet with no direct internet access • Database tier in isolated private subnet accessible only from application tier • Azure Firewall controlling traffic between zones • Bastion host for secure administrative access (no SSH directly to production) Architecture Impact: Multi-tier network security architecture, jump servers for maintenance, strict NSG rules between subnets

<i>System Management Requirement</i>	<i>Impact on Project and how the Technical Architecture will address the impact</i>
Do you have access to the infrastructure required to install and run your own server?	Yes, using Azure cloud infrastructure. Implementation: • Azure Kubernetes Service (AKS) for container orchestration • Azure Virtual Machines for database servers (PostgreSQL, Redis, RabbitMQ) • Full administrative access to configure and manage all resources Architecture Impact: Cloud-native design enabling elastic scaling, automated provisioning, and disaster recovery capabilities
What are the response time targets?	Performance targets defined for critical operations. Response Time Targets: • API endpoints: < 200ms for 95th percentile • Database queries: < 100ms for 90th percentile • Map rendering: < 2 second for initial load (network dependent) • Real-time updates (WebSocket): < 500ms latency • Emergency alert delivery: < 5 seconds end-to-end Architecture Impact: • Redis caching for frequently accessed data (disaster markers, user sessions) • Database indexing strategy on frequently queried columns • CDN for static assets and map tiles • Kubernetes horizontal pod autoscaling based on CPU/memory metrics • Connection pooling to reduce database connection overhead
Availability: - What hours should the service be available? - Is it acceptable to have any scheduled downtime for maintenance? - How important is it that the service be never interrupted, even for unscheduled component failures?	24/7 availability required for emergency response system. Availability Requirements: • Target uptime: 99.9% • Service hours: 24/7/365 with no scheduled maintenance windows • Zero-downtime deployments using Kubernetes rolling updates

- If interruptions do occur, what should be the target time for resuming service?	• Recovery Time Objective (RTO): < 15 minutes for complete system failure • Recovery Point Objective (RPO): < 5 minutes (maximum acceptable data loss) Architecture Impact: • Multi-replica deployment: Minimum 3 replicas per microservice • PostgreSQL master-replica setup with automatic failover • RabbitMQ cluster with mirrored queues • Health checks and liveness probes on all containers • Azure Availability Zones for geographic redundancy • Automated backup and restore procedures
How should partial or total service failures be monitored and handled?	Comprehensive monitoring and alerting system. Implementation: • Azure Monitor for infrastructure-level monitoring • Application Insights for application performance monitoring (APM) • Health check endpoints on all microservices • Runbooks for common incident scenarios Failure Handling: • Graceful degradation: System continues with reduced functionality if non-critical services fail • Automatic service restart for crashed containers • Dead letter queues for failed message processing
Do you need a recovery plan, or will it be covered by existing processes?	Dedicated disaster recovery (DR) plan required. Recovery Plan Components: • Automated PostgreSQL backups: Full backup daily, incremental every 6 hours • Azure Blob Storage snapshots for media files (daily) • DR testing schedule: Quarterly DR drills to validate recovery procedures • Backup retention: 30 days for operational backups, 1 year for compliance • Geographic redundancy: Secondary region (Azure paired regions) for critical data Architecture Impact: Automated backup scripts, monitoring of backup success/failure, documented recovery procedures, separate Azure region for DR failover

Tracking/Documenting: How should the architecture support the process of problem reporting, tracking and fixing? What statistics do you need to keep about the site, and how will they be analysed? What instrumentation should be included in the design to measure performance, response times and availability? Should the architecture include a repository for statistical data?	Comprehensive observability and analytics platform. Problem Tracking: • GitLab Issues for bug tracking and feature requests • Structured logging (JSON format) with correlation IDs for distributed tracing • On-call rotation with PagerDuty for incident management Statistics and Analytics: • User analytics: Active users, disaster reports per day, response times by ERT • System metrics: API call volumes, database query performance, cache hit rates • Business metrics: Average disaster response time, evacuation completion rates • Log Analytics workspace for long-term log retention and analysis Instrumentation: • Request/response timing middleware in FastAPI • Database query performance logging (queries > 100ms) Data Repository: • Azure Log Analytics for centralized log storage (1-year retention) • TimescaleDB (PostgreSQL extension) for time-series metrics
---	---

Client-side Management

Client-side Management Requirement	Impact on Project and how the Technical Architecture will address the impact
Who is the customer? (Internet or Intranet) -- affects browser choice	Public internet users (citizens, emergency responders, government coordinators). Implementation: • Web application supports modern browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+ • Mobile apps for iOS 13+ and Android 8+ (API level 26+) • Progressive Web App (PWA) capabilities for offline access • No legacy browser support (IE11 not supported)

	Architecture Impact: Modern JavaScript (ES2020+), no polyfills for outdated browsers, leveraging latest web standards (WebSocket, Web Crypto API, Service Workers)
What is the level of the user's skill?	Mixed skill levels from basic to advanced. User Profiles: • Citizens: Basic technical skills • Emergency Responders: Moderate skills, need quick access to critical information • Coordinators: Advanced skills, need comprehensive dashboards Architecture Impact: • Simple, intuitive UI with minimal training required • Role-based interfaces: Different views for different user types • In-app tutorials and tooltips for first-time users
What languages should the site support?	• language: English
What are the user's usage patterns? (search or browse)	Both search and browse patterns, context dependent. Usage Patterns: • Citizens: Browse disaster map, search for specific disasters by location • Emergency Responders: Search for assigned tasks, browse disaster details • Coordinators: Search and filter by multiple criteria (status, severity, location) Architecture Impact: • Geospatial search with PostGIS (disasters within radius) • Auto-complete suggestions for search queries • Faceted search filters (severity, type, status, date range) • Recent searches stored in localStorage/mobile storage
How will the application maintain state?	Hybrid state management approach. Implementation: • Client-side state: Redux for application state • Server-side state: JWT tokens for authentication state, Redis for session data • Persistent state: localStorage for preferences, mobile secure storage for tokens

	- Real-time state: WebSocket connections for live updates (disaster status, team locations) Architecture Impact: - Stateless microservices (horizontally scalable) - Sticky sessions not required (load balancer can route freely) - Redux DevTools for debugging state changes - State synchronization between devices when user logs in
Is there a need to distribute application code, and if so, how will it be done?	Yes, distribution via app stores and web hosting. Distribution Channels: - Web app: Hosted on Azure Static Web Apps, distributed via Azure CDN - iOS app: Distributed via Apple App Store - Android app: Distributed via Google Play Store - Over-the-air (OTA) updates for mobile apps Architecture Impact: - Automated CI/CD pipeline for web deployments (build, test, deploy on merge to main) - Mobile app versioning strategy (semantic versioning) - Feature flags for gradual rollout of new features
How will the choice of client affect end-to-end response? (HTML, JavaScript, AJAX, JQuery, VBScript?)	Modern JavaScript framework optimized for performance. Technology Choices: - React with concurrent rendering for better performance - TypeScript for type safety and better developer experience - No jQuery (not needed with modern React) - AJAX replaced with Fetch API and Axios for HTTP requests - Code splitting for faster initial load (React.lazy, dynamic imports) - WebSocket for real-time communication (reduces polling overhead) Performance Impact: - Initial page load: < 5 seconds on 4G connection - Bundle size optimization: < 300KB gzipped for initial bundle - Lazy loading of non-critical components (maps, charts) - Service worker for offline caching (PWA) - Image optimization: WebP format with fallback to JPEG

What are the different user interfaces needed?	Role-based interfaces with shared components. User Interfaces: • Citizen Interface: ○ Disaster reporting form with photo/video upload ○ Interactive map showing nearby disasters ○ Evacuation route navigation ○ Alert notification center • Emergency Responder Interface: ○ Task dashboard with assigned disasters ○ Team coordination and communication tools ○ Real-time location tracking ○ Resource inventory management • Coordinator Interface: ○ Comprehensive disaster management dashboard ○ Team deployment and resource allocation ○ Analytics and reporting tools ○ System-wide monitoring • Admin Interface: ○ User management and permissions ○ System configuration ○ Audit logs and compliance reporting Architecture Impact: Shared component library, role-based routing, responsive design for mobile/tablet/desktop, accessibility compliance
--	---

Network Management

<i>Network Management Requirement</i>	<i>Impact on Project and how the Technical Architecture will address the impact</i>
Will the solution involve the internet? What protocols will be used? (HTTP? HTTPS? FTP? RMI? Messaging? Etc)	Yes, internet-based solution with multiple protocols. Protocols Used: • HTTPS (TLS 1.3) for all REST API communication • WebSocket Secure (WSS) for real-time bidirectional communication • AMQP 0-9-1 for RabbitMQ message queue communication • PostgreSQL wire protocol over encrypted connection • Redis protocol (RESP) over TLS

	No FTP or RMI used (deprecated/insecure) Architecture Impact: All communication encrypted in transit, no plaintext protocols, protocol-specific timeouts and retry logic implemented
What about data, object and application placement? <i>projected transaction volumes, amount of data, interaction?</i>	Strategic placement based on performance and data gravity. Data Placement: - Hot data (active disasters, user sessions): Redis cache (in-memory) - Transactional data: PostgreSQL (SSD-backed, primary region) - Media files: Azure Blob Storage with CDN distribution Projected Volumes: - Daily API requests: ~100,000 (during disasters) - Active concurrent users: 10,000 (peak: 50,000) Application Placement: - Microservices: Kubernetes cluster in primary Azure region - Static assets: Azure CDN (global distribution) - Database replicas: Secondary region for read scalability Architecture Impact: Horizontal scaling for API services, read replicas for database, CDN for global performance, connection pooling to handle high concurrency
What security functions are required/provided by chosen protocol? <i>Level of encryption will affect this, and also performance!</i>	Strong encryption with acceptable performance overhead. Security Functions: - TLS 1.3: Faster handshakes, perfect forward secrecy, modern cipher suites only - WebSocket authentication using JWT tokens passed during connection handshake - Database connection encryption using SSL/TLS Performance Considerations: - TLS 1.3 reduces handshake latency by ~30% vs TLS 1.2 - Session resumption to avoid full handshake on reconnection - Hardware-accelerated encryption (Azure VMs with AES-NI support) - HTTP/2 multiplexing to reduce connection overhead Architecture Impact: Load balancer handles TLS termination (offloads encryption from backend), connection keep-alive to minimize handshakes, monitoring of TLS negotiation times
How does the network affect end-to-end response time?	Network optimization critical for performance. Network Factors:

	• Geographic latency: CDN reduces latency for static content (< 100ms globally) • Bandwidth: Compressed responses (gzip/brotli) reduce transfer time by ~70% • Connection pooling: Reuse TCP connections to avoid handshake overhead • DNS resolution: Azure Traffic Manager for low-latency DNS routing Optimization Strategies: • API response compression (gzip level 6 for balance of speed/size) • Image optimization: Responsive images, WebP format, lazy loading • HTTP/2 server push for critical resources • Edge caching for API responses where appropriate (CDN + Redis) • WebSocket for real-time updates (eliminates polling overhead) Monitoring: • Network latency monitoring between services
--	---

Server-side Management

<i>Server-side Management Requirement</i>	<i>Impact on Project and how the Technical Architecture will address the impact</i>
Single server or multiple servers? Peer-to-Peer? Sensors?	Multiple servers in distributed architecture. Server Architecture: • Multiple Kubernetes nodes (minimum 3 for high availability) • Separate servers for databases (PostgreSQL primary + replica, Redis cluster, RabbitMQ cluster) • Load balancer tier (NGINX running on dedicated nodes) • No peer-to-peer communication (client-server model only) Architecture Impact: Distributed system design, service mesh for inter-service communication
Geographic location for servers?	Primary region with disaster recovery in secondary region. Server Locations: • Primary region: Azure region closest to deployment area • Secondary region: Azure paired region for DR • CDN: Global edge locations for content delivery

	Data Residency: • Compliance with data sovereignty requirements • PII data stored only in approved regions • Backup data replicated to secondary region Architecture Impact: Active-passive DR setup, geo-replication for critical data, Traffic Manager for automatic failover, consideration of cross-region latency for replication
End-user client to server, or server to server required also?	Both client-to-server and server-to-server communication. Client-to-Server: • Mobile/web clients communicate with API Gateway via HTTPS/WSS • JWT-based authentication for all client requests • Rate limiting per user/IP address Server-to-Server: • Microservices communicate via internal network (no internet exposure) • Service-to-service authentication using mutual TLS (mTLS) • Integration with external services (maps, weather, SMS) via HTTPS • Database connections from application tier • Message queue for asynchronous communication between services Architecture Impact: Service mesh for secure service-to-service communication, internal DNS for service discovery, circuit breakers for external service calls
What security functions are required on the server?	Multi-layered server security. Security Functions: • Input validation: All API inputs validated and sanitized • SQL injection prevention: Parameterized queries using SQLAlchemy ORM • XSS prevention: Content Security Policy headers, output encoding • CSRF protection: CSRF tokens for state-changing operations • Rate limiting: Token bucket algorithm, per-user and per-IP limits • Intrusion detection: Azure Security Center for threat detection • Secrets management: Azure Key Vault for API keys, certificates, database passwords • Container security: Image scanning for vulnerabilities, non-root containers Architecture Impact: Security middleware in API gateway, automated security scanning in CI/CD, regular security updates for OS and dependencies, network segmentation with NSGs

How can impact of server on end-to-end response time be estimated, and catered for in the architecture?	Performance modeling and optimization. Estimation Methods: • Load testing with K6 to simulate concurrent users • Performance profiling with cProfile and memory_profiler for Python code • Application Performance Monitoring (APM) with distributed tracing Performance Optimization: • Asynchronous processing: FastAPI with async/await for non-blocking I/O • Database connection pooling: PgBouncer with 100-200 connections • Query optimization: Indexes on frequently queried columns, avoid N+1 queries • Caching strategy: Redis for frequently accessed data, cache TTL based on data volatility • Auto-scaling: Kubernetes HPA based on CPU/memory metrics Capacity Planning: • Server sizing based on load tests (8 vCPU, 16GB RAM per API node) • Database sizing: 4 vCPU, 16GB RAM, 500 IOPS for primary • Monitoring of resource utilization for scaling decisions
---	--

Application Logic

<i>Application Logic Requirement</i>	<i>Impact on Project and how the Technical Architecture will address the impact</i>
Will site use client-side executables? What are their connectivity requirements?	Client-side JavaScript (web) and native code (mobile), no executables. Implementation: • Web: JavaScript/TypeScript running in browser sandbox (no downloads required) • Mobile: Native apps distributed via app stores Connectivity Requirements: • Minimum: 3G connection for basic functionality • Recommended: 4G/WiFi for optimal experience • Offline mode: Core features available without connectivity (view cached disasters, prepare reports) • Progressive Web App: Service worker caches essential resources

	Automatic sync when connection restored Architecture Impact: IndexedDB for offline data storage, background sync API for queued actions, graceful degradation of features based on connectivity
How will application be split between client-side and server-side logic? <i>(affects communications for validation etc/performance?)</i>	Smart client with server-side authority. Client-Side Logic: - UI rendering and user interactions - Form validation (basic checks: required fields, format validation) - Local state management (Redux) - Map rendering and visualization - Offline data caching - Real-time UI updates from WebSocket events Server-Side Logic: - Business logic and workflow enforcement - Authentication and authorization - Data validation (authoritative, cannot be bypassed) - Database operations and transactions - Integration with external services - ML model inference for disaster evaluation - Complex calculations (route optimization, resource allocation) Architecture Impact: - Reduced server load: Client-side validation reduces unnecessary API calls - Better UX: Immediate feedback from client-side validation - Security: Server always re-validates (never trust client) - API design: RESTful endpoints with clear separation of concerns
Additional access security required?	Yes, comprehensive access control beyond authentication. Security Measures: - Role-Based Access Control (RBAC): Four roles with distinct permissions - API endpoint authorization: Decorators check user roles before execution - Resource-level permissions: Users can only access their own data or assigned disasters - Geographic access control: Coordinators restricted to their assigned regions

	• Time-based access: Emergency response tokens expire after incident resolution • IP whitelisting for admin functions Architecture Impact: Authorization middleware in all microservices, permissions stored in database and cached in Redis, audit logging of all access attempts, regular access reviews
--	---

Connectors

<i>Connector Requirement</i>	<i>Impact on Project and how the Technical Architecture will address the impact</i>
What external systems, applications and (sensor) data does your project need to access?	Multiple external integrations required. External Systems: • Mapping Services: Valhalla routing engine (self-hosted), Google Maps API (backup) • Weather Service: OpenWeatherMap API for current conditions and forecasts • SMS Gateway: Twilio for OTP and emergency alerts • Email Service: SendGrid for transactional emails Architecture Impact: Dedicated External Integration Service as facade for all external calls, adapter pattern for different APIs, circuit breakers to prevent cascade failures
How should data be transferred between different systems?	RESTful APIs with structured data formats. Data Transfer Methods: • REST APIs: JSON payloads over HTTPS for most integrations • WebSocket: Real-time data streaming • Message Queue: Asynchronous processing of external data • Batch transfers: Scheduled jobs for large data exports/imports (Azure Data Factory) Data Formats: • JSON for API communication (standardized schemas) • GeoJSON for geographic data Architecture Impact: API versioning strategy (v1, v2 in URL), data transformation layer, schema validation using Pydantic
How current does the information have to be? Use caches?	Varying freshness requirements based on data type. Real-time Data (no caching):

	• Disaster status updates • Emergency team locations • Incoming disaster reports Near-real-time (1-5 minute cache): • Weather data (updated every 5 minutes) • Traffic conditions (updated every 2 minutes) Cacheable Data (15-60 minute cache): • Map tiles and base layers • Historical disaster data • User profiles and permissions Static Data (24 hour cache): • Geographic boundaries • System configuration Architecture Impact: Redis with TTL-based expiration, cache invalidation strategy for updated data, cache-aside pattern, separate cache namespaces for different data types
Is synchronous or asynchronous access required? <i>Off-line, OK?</i>	Hybrid approach based on operation criticality. Synchronous Operations: • User authentication and authorization • Data retrieval for immediate display • Critical updates (disaster status changes) • Route calculation for immediate navigation Asynchronous Operations: • Media file uploads (photos/videos) - processed in background • Email notifications - queued and sent via RabbitMQ • ML model predictions - job queue for batch processing • External API calls (weather updates, traffic data) • Report generation and analytics Offline Capabilities: • View cached disaster information • Prepare disaster reports (submitted when online) • View evacuation routes (requires pre-cached map data) • Background sync queues pending actions Architecture Impact: Message queue for async tasks, job status tracking, offline queue in client apps, conflict resolution for offline edits

Is access to different operating systems, network protocols, application environments required? which connector? CICS? MQSeries? RPC?	Modern integration patterns, no legacy connectors. Cross-Platform Access: • Operating systems: Windows, macOS, Linux servers; iOS and Android clients • All communication via standard internet protocols (HTTPS, WSS) • No mainframe integration (CICS) required • RabbitMQ used instead of IBM MQSeries • REST APIs instead of RPC/SOAP Integration Patterns: • API Gateway pattern for unified entry point • Adapter pattern for external service integration • Event-driven architecture for loose coupling • Service mesh for inter-service communication Architecture Impact: Platform-agnostic design, containerization ensures consistency across environments, standard protocols enable easy integration
Are additional security policies required?	Yes, security policies for external integrations. Security Policies: • API key rotation: External service keys rotated every 90 days • Secrets management: All credentials stored in Azure Key Vault, never in code • Network isolation: External API calls through dedicated egress gateway • Data filtering: Sensitive data removed before sending to external services • Rate limiting: Prevent abuse of external services (cost control) Architecture Impact: Centralized secrets management, egress filtering, API call monitoring, webhook verification middleware, compliance documentation
Can scalability and performance requirements be predicted, and how will the project address these?	Yes, predictable patterns with surge capacity. Usage Predictions: • Normal operations: ~1,000 concurrent users, ~200 req/sec • Major disaster event: ~50,000 concurrent users, ~5,000 req/sec • Peak periods: Immediate aftermath of disasters (first 6 hours) • Growth projection: 50% annual increase in user base Scalability Strategy:

	• Horizontal Scaling: ○ Kubernetes autoscaling: Scale from 3 to 50 pods per service based on CPU/memory ○ Database read replicas for query load distribution ○ Redis cluster sharding for cache scalability • Performance Optimization: ○ CDN for static content and media files ○ Database query optimization and indexing ○ Asynchronous processing for non-critical operations ○ Connection pooling and keep-alive • Capacity Planning: ○ Quarterly load testing to validate scaling ○ Pre-warming during anticipated disasters (hurricane forecasts) ○ Reserved capacity in cloud for cost optimization Architecture Impact: Stateless microservices enable horizontal scaling, monitoring and alerting for capacity issues, automated scaling policies, cost analysis and optimization
--	---